Name: Melvina Kosisochukwu Anudu

Reg No: 2024/B/CSC/0331

Date: October 8, 2026

1. Executive Summary

This project simulates the end-to-end design and implementation of a scalable information management system for a retail enterprise, modeled after Kamsberry Store. The objective is to apply core data management principles: data collection, structured storage, relational indexing, query retrieval, and search algorithm optimization to build an efficient data pipeline.

2. Assessment Objectives & Scope

The primary goals of this assessment are:

1. **Data Capture & Representation:** Collect raw customer transaction records through an interactive user interface and format them into a structured flat file.
2. **Data Indexing & Database Implementation:** Relational schema design, structured data importing, and performance optimization using primary, secondary, and non-clustered indexing techniques.
3. **Information Retrieval:** Writing and executing structured queries to extract specific transaction subsets based on categorical and time-series criteria.
4. **Search Algorithm Performance Benchmarking:** Comparing linear and logarithmic search techniques on indexed versus unindexed dataset configurations to measure real-world retrieval efficiency.

3. Project Implementation & Methodology

Step 1: Data Capture & Representation

To capture retail transactions cleanly, a Google Forms survey was designed and deployed under the name **Kamsberry Store RESPONSES**.

- Data fields captured include: Customer Name, Purchase Date, Product ID, Product Category, and Transaction Amount.
- Form responses were exported into a structured tabular Comma-Separated Values (.csv) dataset.
- A total dataset containing 30 unique transaction records was assembled for analysis and database ingestion.

Step 2: Data Indexing & Relational Database Management

Database Creation & Data Import
A relational table named retail_purchases was created to store transaction records. The imported records were structured under six key attributes: customer_id, customer_name, purchase_date, product_id, product_category, and amount.

Index Implementation
To maximize retrieval speed and eliminate full table scans during querying, three types of indexes were configured:

1. **Primary Index (**customer_id**):** Configured as an auto-incrementing integer primary key to serve as the default clustered index for row identification.
2. **Secondary Index (**product_category**):** Created on product_category (idx_category) to group identical category records together, accelerating categorical lookups.
3. **Non-Clustered Index (**purchase_date**):** Applied on purchase_date (idx_date) to optimize range-based filtering and chronological sales reporting.

Query Execution & Information Retrieval
Two core retrieval tasks were conducted to verify index responsiveness:

- **Category Filtering:** Executed a query filtering transactions belonging strictly to the **Electronics** category, ordered from highest to lowest purchase amount. The query successfully returned 11 relevant records sorted in descending order of value.
- **Date-Range Filtering:** Executed a query filtering transactions occurring within **September 202**, ordered chronologically from earliest to latest.

Step 3: Search Simulation & Performance Analysis
To evaluate how indexing and sorting directly impact algorithmic performance, two distinct search strategies were implemented in Python to locate product ID PROD101 across the dataset:

1. **Sequential Search (Unindexed Data):**
    - Scans every record sequentially from start to finish until all matches are identified.
    - Operates at $O(N)$ linear time complexity.
2. **Binary Search (Indexed / Pre-Sorted Data):**
    - Uses a divide-and-conquer strategy on pre-sorted product data.
    - Operates at $O(\log N)$ logarithmic time complexity.

4. Key Results & Performance Benchmark

Search Performance Benchmark Summary
The performance metrics captured during execution are summarized below:

- **Sequential Search (Unindexed):**

    - Matches Found: 4
    - Total Comparisons: **30 comparisons**
    - Elapsed Time: $0.0066ms$

- **Binary Search (Indexed / Sorted):**

    - Matches Found: 4
    - Total Comparisons: **3 comparisons**
    - Elapsed Time: $0.0076ms$

Performance Findings

- **90% Reduction in Operations:** Binary search reduced the required comparison operations from 30 down to 3, demonstrating the power of pre-sorting and indexing.
- **Scalability:** On small sample sizes ($N = 30$), microsecond time differences reflect minor system execution overhead rather than algorithmic speed. However, as data scales into thousands or millions of entries, $O(\log N)$ indexed lookups prevent system bottlenecks and maintain fast query response times.

5. Summary of Deliverables

The project deliverables prepared for submission include:

1. **Dataset (**.csv**):** Kamsberry Store RESPONSES - Form Responses 1.csv containing captured transaction data.

2. **SQL Script (**.sql**):** Database.sql containing table schema definitions, index creation steps, and filtering queries.

3. **Python Script (**.py**):** Search.py containing search algorithms and performance benchmark counters.

4. **Execution Screenshots:** Visual proofs confirming SQL query outputs and terminal execution results.

5. **Project Report (**.pdf**):** This comprehensive documentation of project workflow, methodology, and performance findings.
6. **Google Form link:** https://docs.google.com/forms/d/e/1FAIpQLSdNa42jelAmEDaf5ByqzD2OtE8JpeXBQzNljbzN1jnDoCLE2Q/viewform?usp=dialog

6. Conclusion

This assessment demonstrates the vital role that structured database design, indexing strategies, and algorithm

selection play in modern information management systems. Applying primary, secondary, and non-clustered indexes transforms unstructured search operations into efficient, scalable data retrievals suitable for real-world enterprise application.